Experiment No. 02
TCP/IP Utility Commands

Aim

To study, implement, and analyze commonly used TCP/IP utility commands.

Theory

TCP/IP (Transmission Control Protocol/Internet Protocol) is the standard communication protocol used for networking and the Internet. Various TCP/IP utility commands help users diagnose network connectivity, verify configurations, resolve domain names, and trace communication paths.

Common TCP/IP Utility Commands

No.	Command	Purpose
1	ping	Checks connectivity between two hosts
2	ipconfig (Windows) / ifconfig or ip addr (Linux)	Displays IP configuration
3	tracert (Windows) / traceroute (Linux)	Displays the route packets take to a destination
4	nslookup	Finds the IP address associated with a domain name
5	netstat	Displays active network connections and listening ports
6	arp	Displays and modifies the ARP cache
7	hostname	Displays the system's host name
8	route	Displays and modifies the routing table

Conclusion:

Hence, we have implemented and analyze TCP /IP utility commands successfully.

Experiment No. 03
TCP and UDP Traffic Generation

Aim

To generate and analyze TCP and UDP traffic using Cisco Packet Tracer in Simulation Mode.

Theory

The Transport Layer of the TCP/IP model provides end-to-end communication between devices. It mainly uses two protocols: TCP and UDP.

Transmission Control Protocol (TCP)

TCP is a connection-oriented transport layer protocol that establishes a connection before transmitting data. It provides reliable communication by using acknowledgements, sequencing, flow control, and error recovery.

Before data transmission, TCP establishes a connection through a **Three-Way Handshake** (SYN, SYN-ACK, ACK). After the connection is established, data is transferred and the session is terminated using FIN and ACK packets.

Applications using TCP

- HTTP
- HTTPS
- FTP
- SMTP
- POP3

User Datagram Protocol (UDP)

UDP is a connectionless transport layer protocol. It sends data without establishing a connection, making communication faster but without guaranteeing delivery or packet ordering. Since there is no connection establishment or acknowledgement, UDP has less overhead than TCP.

Applications using UDP

- DNS
- DHCP
- VoIP
- Online Gaming
- Video Streaming

TCP vs UDP at a Glance

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Acknowledgements, sequencing, error recovery	No delivery or ordering guarantee
Overhead	Higher	Lower
Speed	Slower	Faster

Conclusion:

TCP and UDP traffic were successfully generated and analyzed using Cisco Packet Tracer. UDP traffic was observed during DNS communication, while TCP traffic was observed during web browsing through connection establishment, data transfer, and connection termination

Experiment No. 06

Static Routing Using GNS3

Aim

To design and implement static routing using two routers and two PCs in GNS3, and verify connectivity between the PCs.

Theory

Static routing is a routing technique in which the network administrator manually adds routes to the router's routing table. The router uses these manually configured routes to forward packets to networks that are not directly connected.

Networks used

- Network 1: 192.168.1.0
- Network 2: 192.168.2.0
- Router-to-router network: 10.0.12.0

Since R1 does not directly know about Network 2 and R2 does not directly know about Network 1, static routes are configured manually.

Static Route Syntax

```
ip route <destination-network> <subnet-mask> <next-hop-IP>
```

Example on R1:

```
ip route 192.168.2.0 255.255.255.0 10.0.12.2
```

This means R1 should forward packets destined for 192.168.2.0 to R2 at 10.0.12.2.

Addressing Table

Device	Interface	IP Address	Subnet Mask	Default Gateway
PC1	Ethernet	192.168.1.2	255.255.255.0	192.168.1.1
R1	G0/0	192.168.1.1	255.255.255.0	—
R1	G0/1	10.0.12.1	255.255.255.252	—
R2	G0/0	10.0.12.2	255.255.255.252	—
R2	G0/1	192.168.2.1	255.255.255.0	—
PC2	Ethernet	192.168.2.2	255.255.255.0	192.168.2.1

Conclusion:

Static routing was successfully configured between two routers and two PCs in GNS3. Connectivity between PC1 and PC2 was successfully verified using the ping command

Experiment No. 07

Dynamic Routing (EIGRP) Using GNS3

Aim

To design and configure dynamic routing using EIGRP (Enhanced Interior Gateway Routing Protocol) in GNS3 with 2 routers and 2 PCs, and verify communication between the two different networks.

Theory

EIGRP is a dynamic routing protocol developed by Cisco. It is an advanced distance-vector routing protocol that uses the DUAL (Diffusing Update Algorithm) to calculate the best path to a destination.

EIGRP uses a composite metric primarily based on bandwidth and delay. It allows routers to automatically learn remote networks and update their routing tables when there is a change in the network.

Advantages of EIGRP

- Fast convergence
- Supports VLSM and CIDR
- Uses bandwidth and delay for route selection
- Maintains neighboring router information
- Supports automatic route summarization
- Suitable for medium and large networks

Addressing Table

Device	Interface	IP Address	Subnet Mask
PC1	Ethernet	192.168.1.10	255.255.255.0
R1	G0/0	192.168.1.1	255.255.255.0
R1	G0/1	10.0.0.1	255.255.255.252
R2	G0/1	10.0.0.2	255.255.255.252
R2	G0/0	192.168.2.1	255.255.255.0
PC2	Ethernet	192.168.2.10	255.255.255.0

Conclusion:

Thus, dynamic routing using EIGRP was successfully designed and configured in GNS3 using 2 routers and 2 PCs. The routers exchanged routing information dynamically through EIGRP, and connectivity between PC1 and PC2 was successfully verified using the ping command.

Experiment No. 08

Client-Server Communication Using Python Sockets

Aim

To create a simple socket program in Python for exchanging basic information between a server and a client.

Theory

What is Socket Programming? A socket is an endpoint used for communication between two computers or two programs over a network. Socket programming allows a client and a server to communicate with each other by sending and receiving data.

Client-Server Model Socket communication generally follows the client-server model.

Server steps

1. Creates a socket.
2. Assigns an IP address and port.
3. Waits for a client.
4. Accepts the connection.
5. Receives and sends information.
6. Closes the connection.

Client steps

1. Creates a socket.
2. Connects to the server.
3. Sends information.
4. Receives the server's response.
5. Closes the connection.

TCP Socket In this practical, we use TCP (Transmission Control Protocol), which provides:

- Reliable communication
- Connection-oriented communication
- Ordered delivery of data
- Error checking

- Data retransmission when required

Conclusion:

The Python socket program was successfully executed, and basic information was exchanged between the client and server using TCP socket communication.

Experiment No. 09

File Transfer Using UDP Sockets

Aim

To write and execute a program using UDP sockets to enable file transfer between two machines over a network.

Theory

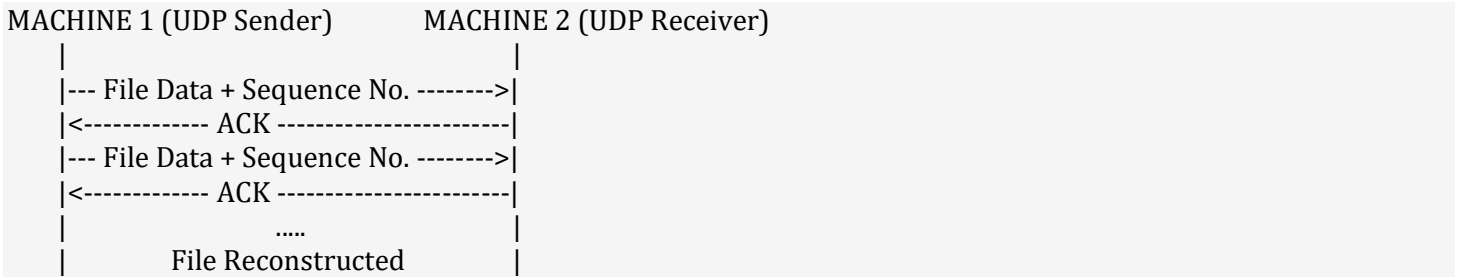
UDP (User Datagram Protocol) is a connectionless transport-layer protocol. It provides a simple method for sending datagrams between two applications without establishing a dedicated connection.

For file transfer using UDP

- The sender/client reads the file in small chunks.
- Each chunk is converted into a UDP datagram.
- The datagram is sent to the receiver using the receiver's IP address and port number.
- The receiver/server receives the datagrams and writes the data into a new file.

Unlike TCP, UDP does not guarantee delivery, ordering, or duplicate protection. Therefore, for a practical implementation, the program can use sequence numbers and acknowledgements to improve reliability.

Basic Working



Important UDP Functions

Function	Purpose
socket()	Creates a UDP socket
bind()	Assigns IP address and port to socket
sendto()	Sends UDP data
recvfrom()	Receives UDP data
close()	Closes the socket

Conclusion:

Thus, the program demonstrates file transfer between two machines using UDP sockets. UDP provides fast and connectionless communication, but it does not inherently guarantee reliable delivery or packet ordering. The use of sequence numbers, acknowledgements, and retransmission makes the practical implementation more reliable.

Experiment No. 10

Analyze Captured Packets for Potential Security Issues

Aim

To capture and analyze network packets using Wireshark and identify potential security issues such as unauthorized access attempts, clear-text credentials, suspicious communication, and other malicious network activities.

Theory

Packet analysis is the process of examining network packets exchanged between devices to understand network communication and identify abnormal or suspicious activities.

Wireshark is a network protocol analyzer that captures packets from a network interface and provides detailed information about protocols, IP addresses, ports, packet contents, and communication patterns.

Security Issues Identifiable During Packet Analysis

1. Clear-text credentials

- Some protocols transmit usernames and passwords without encryption.
- Examples: HTTP, FTP, Telnet, and some legacy protocols.
- Such credentials can potentially be exposed to anyone able to capture the traffic.

2. Unauthorized access attempts

- Repeated connection attempts to different ports or services may indicate port scanning or unauthorized access attempts.
- Repeated failed authentication attempts can also be suspicious.

3. Unencrypted communication

- HTTP, FTP, and Telnet traffic can expose sensitive information.
- HTTPS and SSH provide encryption for their application traffic.

4. Suspicious IP addresses

- Communication with unexpected or unknown external systems should be investigated.
- An IP address alone, however, is not sufficient evidence of malicious activity.

5. Port scanning

- A host attempting connections to many ports or many systems in a short period may indicate scanning activity.

6. Abnormal traffic patterns

- Large numbers of packets, repeated connection failures, unusual protocols, or unexpected data transfers can indicate potentially suspicious behavior

Conclusion:

The captured packets were successfully analyzed using Wireshark/TShark. Network protocols, IP addresses, ports, authentication-related traffic, TCP connection attempts, DNS queries, and other communication patterns were examined to identify potential security issues.